

Report conversation

この2つの処理は一緒でしょうか？

```
const convertHankakuToZenkaku = (text: string) => {
  // " "などの目に見えない文字をまとめて全角スペース
  (0x3000)に置き換える
  text = text.replace(/\p{Z}/gu, () =>
String.fromCharCode(0x3000));

  // "!"から"~"までの範囲の文字(数字やアルファベット)を
  全角に置き換える
  return text.replace(/[\u0021-\u007e]/g, (s) => {
    return String.fromCharCode(s.charCodeAt(0) + 0xfee0);
  });
};
```

```
/// ASCII文字を全角文字に変換する。
fn to_zenkaku(surface: &str) -> String {
  // 元実装：
https://github.com/VOICEVOX/voicevox/blob/69898f5dd001d28d4de355a25766acb0e0833ec2/src/components/DictionaryManageDialog.vue#L379-L387
  let mut result = String::new();
  for c in surface.chars() {
    let i = c as u32;
    result.push(if (0x21..=0x7e).contains(&i) {
      char::from_u32(0xfee0 + i).unwrap_or(c)
    } else {
      c
    });
  }
  result
}
```

はい、これらの2つの関数は基本的に同じ処理を行っています。両方ともASCIIの半角文字を全角文字に変換しています。ただし、いくつかの違いがあります。

1. JavaScriptの関数 `convertHankakuToZenkaku` は、最初に全ての空白文字を全角スペースに置き換えています。一方、Rustの関数 `to_zenkaku` はこの処理を行っていません。
2. JavaScriptの関数は、正規表現を使用して半角文字を全角に変換しています。一方、Rustの関数は、文字列を文字ごとに反復処理し、各文字を全角に変換しています。
3. JavaScriptの関数は、変換不能な文字があった場合にはそのままの文字を返します。一方、Rustの関数は、変換不能な文字があった場合には元の文字をそのまま返します。

これらの違いを除けば、両方の関数は同じ目的を達成するために作られており、基本的に同じ処理を行っています。